

PRACTICAL 21 -

Objective - Given a string containing only parentheses '()', determine if the input string is valid (i.e., open brackets are closed in the correct order).

CODE -

```
1  #include <stdio.h>
2  #include <stdbool.h>
3  #include <string.h>
4  #include <stdlib.h>
5
6  // Structure for a stack
7  struct Stack {
8      int top;
9      unsigned capacity;
10     char* array;
11 };
12
13 // Function to create a stack
14 struct Stack* createStack(unsigned capacity) {
15     struct Stack* stack = (struct Stack*) malloc(sizeof(struct Stack));
16     stack->capacity = capacity;
17     stack->top = -1;
18     stack->array = (char*) malloc(stack->capacity * sizeof(char));
19     return stack;
20 }
21
22 // Function to check if the stack is empty
23 bool isEmpty(struct Stack* stack) {
24     return stack->top == -1;
25 }
26
27 // Function to push an element onto the stack
28 void push(struct Stack* stack, char item) {
29     stack->array[++stack->top] = item;
30 }
31
32 // Function to pop an element from the stack
33 char pop(struct Stack* stack) {
34     if (!isEmpty(stack)) {
35         return stack->array[stack->top--];
36     }
37     return '\0'; // Return null character for empty stack
38 }
39
40 // Function to check if two characters are matching parentheses
41 bool isMatchingPair(char character1, char character2) {
```

```
42     if (character1 == '(' && character2 == ')')
43         return true;
44     if (character1 == '{' && character2 == '}')
45         return true;
46     if (character1 == '[' && character2 == ']')
47         return true;
48     return false;
49 }
50
51 // Function to check if the given string of parentheses is valid
52 bool isValid(char* exp) {
53     // Create a stack of sufficient capacity
54     struct Stack* stack = createStack(strlen(exp));
55     int i;
56
57     // Iterate through the input string
58     for (i = 0; exp[i]; i++) {
59         // If the character is an opening parenthesis, push it onto the
           stack
60         if (exp[i] == '(' || exp[i] == '{' || exp[i] == '[')
61             push(stack, exp[i]);
62
63         // If the character is a closing parenthesis
64         if (exp[i] == ')' || exp[i] == '}' || exp[i] == ']') {
65             // If the stack is empty, there is no matching opening
               parenthesis
66             if (isEmpty(stack))
67                 return false;
68
69             // Pop the top element from the stack
70             char poppedChar = pop(stack);
71
72             // Check if the popped character and the current character
               are a matching pair
73             if (!isMatchingPair(poppedChar, exp[i]))
74                 return false;
75         }
76     }
77
78     // If the stack is empty after processing the entire string, then
       all parentheses are balanced
79     return isEmpty(stack);
80 }
81
82 int main() {
83     char exp[100];
84
85     printf("Enter a string containing only parentheses: ");
86     scanf("%s", exp);
87
88     if (isValid(exp))
```

```
89     printf("True\n");
90     else
91         printf("False\n");
92
93     return 0;
94 }
```

OUTPUT -

```
Enter a string containing only parentheses: [{(())}]
False
Enter a string containing only parentheses: [{(())}]
True
```